Shwetha C

Advanced Problem-Solving Lab – High-IQ Track

Activity H1 – Solve the "Hidden Problem" Behind the Stated Problem

Title: From Surface Problem to Real Problem

Goal

Train yourself to see the *real* problem, not just the stated one.

Instructions

You are given a "client statement":

"Our exam system is too slow. We want a Python script to speed it up."

1. Identify at least 3 possible *real* problems this statement might hide.
   - Hidden problem 1: Insfficient database queries
   - Hidden problem 2: Poor algorithm design
   - Hidden problem 3: server overload or hardware limitation
2. For each hidden problem, state:
   - What data would you need to confirm it?
     - Problem 1: Query execution time, database logs.
     - Problem 2: Code performance metrics, runtime analysis
     - Problem 3: CPU usage, memory usage, server logs.
3. Choose ONE hidden problem and re-frame it as a clear technical problem statement (2–3 lines):

New problem statement:

The system is slow due to insfficient database queries causing delay in fetching results. optimize queries to reduce response time.

4. Design a very small Python experiment that could validate or reject your reframed problem (pseudo-code is fine):

python

```
# Experiment idea (what will you measure, on what data, how?)
# measure time taken by current query vs optimized query.
start time = time.time()
run_query()
end_time = time.time()
print("Execution time:", end_time - start_time)
```

Activity H2 – Multiple Competing Solutions

Title: Design 3 Solutions Before Coding 1

Goal
Avoid fixation. Explore at least three distinct solution strategies.

Instructions
Problem:

"You receive a stream of log lines (text). You must detect 'suspicious' activity in real time."

1. Give three different solution ideas, each using a distinct main technique or data structure.
   - Approach A (e.g., rule-based / regex / thresholds):

Rule - based keyword matching

   - Approach B (e.g., statistical / sliding window / counters):

sliding window count of suspicious events

   - Approach C (e.g., simple ML / anomaly detection concept):

Simple anomaly detection using ML

2. For each approach, list pros and cons (at least one each).
   - Approach A – Pros: simple/fast Cons: Limited accuracy
   - Approach B – Pros: Better detection Cons: More computation
   - Approach C – Pros: Adaptive Cons: complex
3. Choose one approach to implement first and justify:

I choose approach A *because*

It is simple and easy to implement first.

4. Sketch high-level Python structure (modules/classes, not full code):

python

 # Modules / classes / key functions list

Functions:
read_logs()
detect_suspicious()
alert_user()

Activity H3 – Self-Constrained Challenge: "Elegant Code Only"

Title: Solve with Elegance (Clarity, Minimalism, Correctness)

Goal

Produce code that is not only correct, but also minimal and beautiful.

Problem

Implement a mini "pipeline" for processing a list of tasks:

- Each task has an ID, description, and status ("todo", "doing", "done").
- You must:
  - Load tasks from a JSON file.
  - Filter by status.
  - Sort by ID.
  - Print a clean summary.

Constraints (you impose on yourself)

1. Maximum 25 lines of *real* Python (excluding blanks and comments).
2. No function longer than 7 lines.
3. Names must be meaningful and short.

Steps

1. Design your data model in 2–3 lines:

list of dictionaries with ID, description, status

2. Decide at least one thing you will *not* do to keep it minimal:

Avoid complex classes to keep code simple.

3. Write your code here (respecting constraints): import json
load tasks

python

 # Elegant pipeline solution

4. Reflection:
   - What did you have to sacrifice (if anything) to keep elegance?

Reduced features to keep code short + clean

code =
```
import json
#load tasks
with open("tasks.json") as f:
  tasks = json.load(f)
# Filter+sort
tasks=[t for t in tasks if
t["status"]==="todo"]
tasks = sorted(tasks, key=lamda x: x["id"])
# print summary
for t in tasks:
  print(t["id"], "-",
t["description"], "(", t["status"], ")")
```

Activity H4 – Complexity and Scaling Thought Experiment

Title: How Will Your Solution Break at Scale?

Goal
Develop realistic intuition about performance and scaling.

Instructions
Take ANY non-trivial function or algorithm you have recently written (search, summarization, scoring, etc.).

1. Describe what it does, in *one line*:

   Search for an element in a list

2. Estimate its time complexity in Big-O notation, and justify in 1–2 lines:
   - Big-O: O(n)
   - Because: it checks each element

3. Scaling scenarios – imagine input sizes:
   - $n = 10^3$ (1,000)
   - $n = 10^6$ (1,000,000)
   - $n = 10^9$ (1,000,000,000)

For each, predict: will it still be usable (Y/N) and why?

n	Usable (Y/N)	Reason (1 short sentence)
10^3	yes	Fast
10^6	May be/yes	slower
10^9	No	Too slow

4. Suggest one change to improve scalability and briefly describe how it helps:

   use a set for faster look up (O(1)).

Activity H5 – Adversarial Thinking: Break Your Own Design

Title: Attack Your Own Program

Goal

Develop robustness and defensive mindset.

Instructions

Choose one of your own projects or mini-tools (file reader, API client, CLI, etc.).

1. Write a short description of what it is supposed to do:

   Reads file & processes data

2. Now wear the hat of an *attacker* or malicious user.
   List 5 ways to misuse / break it:

   - Attack 1: Empty file
   - Attack 2: Invalid data
   - Attack 3: Very Large file
   - Attack 4: wrong file format
   - Attack 5: Malicious input

3. For each attack, suggest one defence in plain language:

   - Defence for attack 1: Check file not empty.
   - Defence for attack 2: Validate input
   - Defence for attack 3: limit file size
   - Defence for attack 4: Check format
   - Defence for attack 5: Sanitize input

4. Implement at least one of these defences in your code and paste the updated snippet:

python

 # Hardened code snippet:

```
if not data:
    print ("File is Empty")
try:
    value = int (data)
Except:
    print(" Invalid input")
```

Activity H6 – Cross-Domain Transfer

Title: Apply Known Pattern to a New Domain

Goal

Practice lifting a known idea/pattern from one domain to a different one.

Instructions

Pick a pattern you already know well, e.g.:

- Sliding window
- Two-pointer technique
- Dynamic programming table
- Producer–consumer
- Map–reduce style
- Pipeline of filters

1. Name of pattern: sliding window
2. Original domain where you learned it (e.g., arrays, strings):

   Arrays

3. New domain to apply it in (e.g., logs, time series, learning activities, medical data):

   log data

4. Describe how you would adapt the pattern to this new domain, in 5–8 lines (can include pseudo-code):

   use sliding window to track recent logs & detect frequent events in time window

5. Optional: implement a small prototype showing this transfer:

python

```
# Pattern transfer prototype
window = logs[-5:]
for log in window:
    process(log)
```

Activity H7 – Meta-Cognition: Your Personal Problem-Solving Manual

Title: Write Your Own "How I Solve Problems" Guide

Goal

Turn your tacit skills into an explicit, reusable system.

Instructions

1. List the first three things you usually do when you get a new problem (be honest):
   - Step 1: Read problem
   - Step 2: Think solution idea
   - Step 3: Start coding.
2. Now, design your ideal 6–8 step process (what a "future you" would always do):

1. understand the problem clearly
2. Identify inputs + outputs
3. Break problem into smaller steps
4. choose approach (logic/ algorithm)
5. write code
6. Test with example
7. (optional) Fix errors (if any)
8. (optional) Improve + optimize

3. For each step, write one question you will ask yourself:
   - Step 1 question: what is the problem asking?
   - Step 2 question: what input + ouplits needed?
   - Step 3 question: can I divide the problem in steps?
   - Step 4 question: what is the best way to solve it?
   - Step 5 question: Is my code correct?
   - Step 6 question: Does it work for all test cases?
4. Final instruction to yourself (one sentence):

"When I feel stuck, I will break the problem into smaller steps + solve one by one".